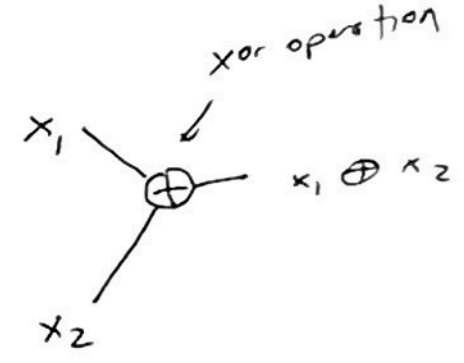
XOR = exclusive OR

x_1 \ x_2	false	true
false	F	T
true	T	F

Figure 1: Truth Table



or



For this example assume we are reading analog values subjected to noise. we let 0 denote false and 1 denote true

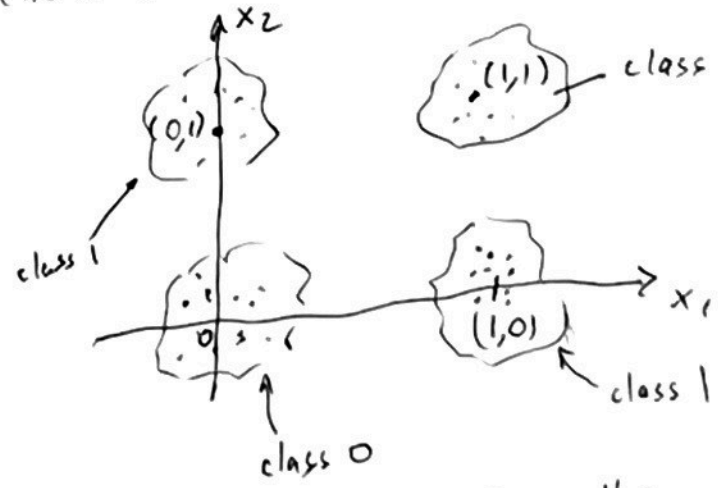


Figure 2: xor scatter

How would we represent an XOR using logistic regression?

- class 0 and class 1 are not linearly separable

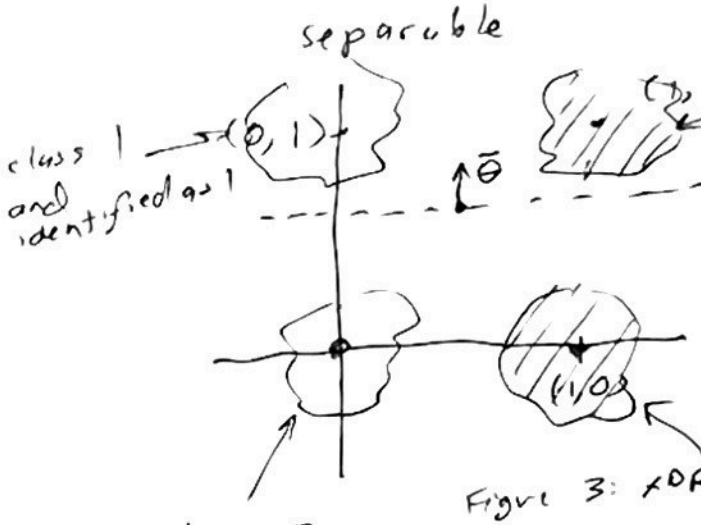


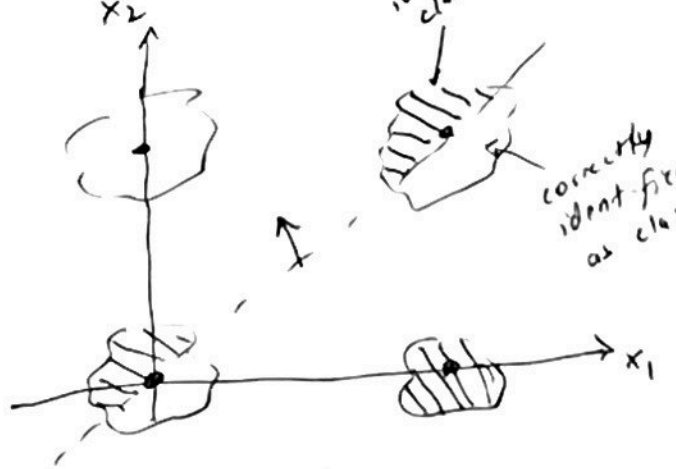
Figure 3: XOR w/ horizontal decision boundary

(x_1, x_2)	$x_1 \oplus x_2$ actual	identified as	predicted correct $x_1 \oplus x_2$?
0,0	0	0	1
0,1	1	1	1
1,0	1	0	0
1,1	0	1	0

identified as class 1, but 0

50% accuracy

//// denotes incorrect



accuracy ≈ 50%

decision boundary

Figure 4: XOR with diagonal decision boundary

No matter how I draw a linear decision boundary accuracy cannot reach 100%

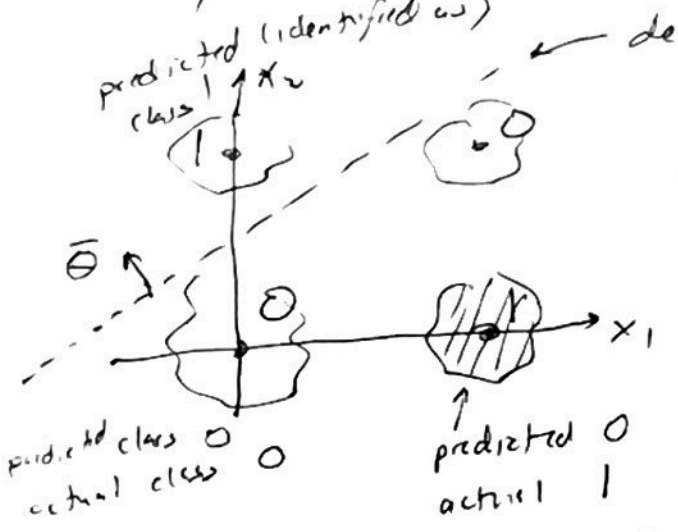


Figure 5: XOR w/ 75% accuracy

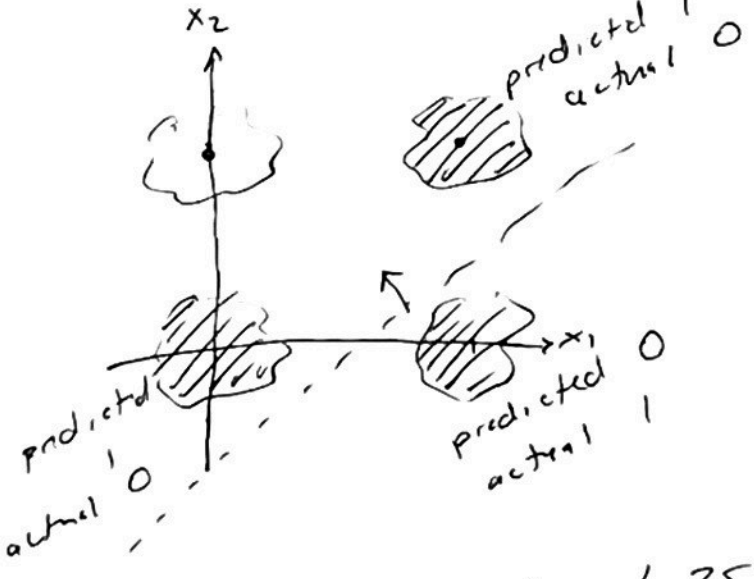


Figure 5: XOR w/ 25% accuracy

How can we solve this problem?

We can't so long as we have a single linear decision boundary.

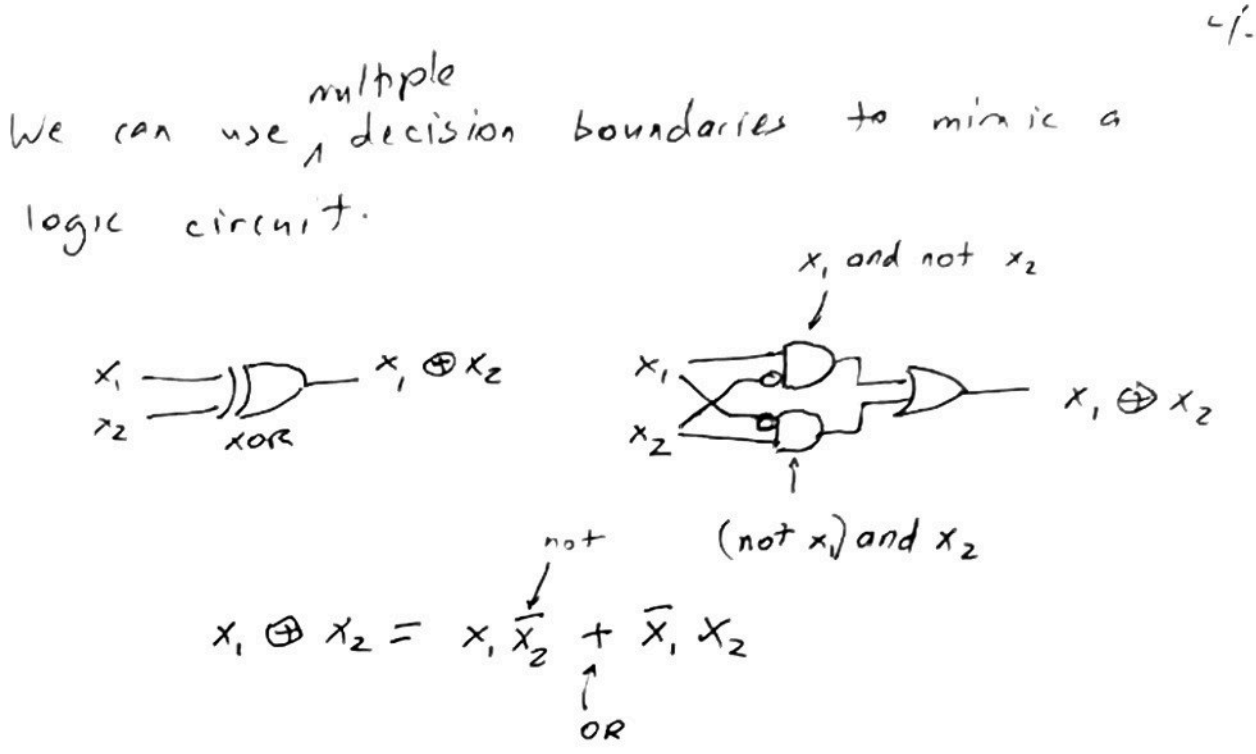


Figure 6: XOR using logic gates

We can implement each gate in Figure 6 as a linear boundary. Hidden node 1 computes x_1 **and not** x_2: it fires when the difference $x_1 - x_2$ exceeds a threshold. Hidden node 2 computes (**not** x_1) **and** x_2: it fires when $x_2 - x_1$ exceeds the same threshold. The output node ORs them.

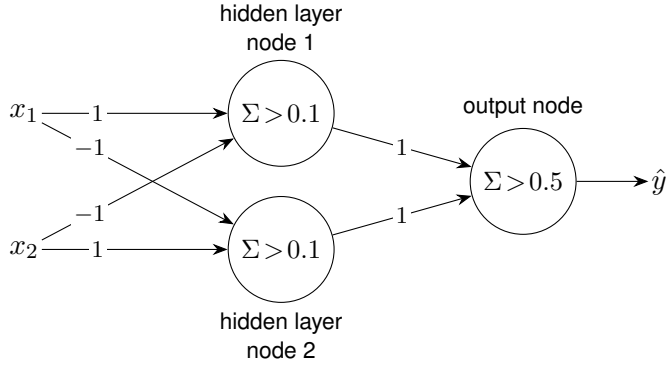


Figure 7: XOR represented using weights and thresholds: one hidden layer of two nodes.

Each node in Figure 7 can be implemented as its own logistic regression. This is our 1st neural network.

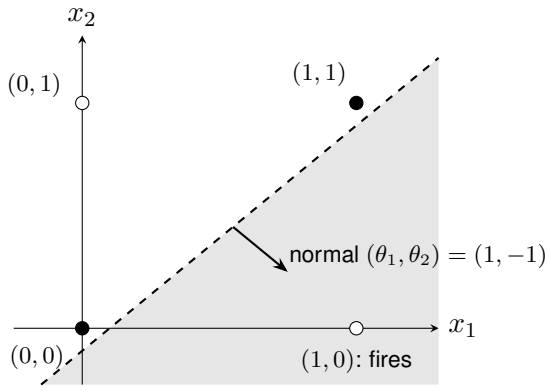


Figure 8: Hidden layer node 1 implements x_1 **and not** x_2, i.e. $x_1 - x_2 > 0.1$. $\bar{\theta} = (-0.1,\ 1,\ -1)$; the decision boundary (dashed) resides at $\bar{x}^T\bar{\theta} = 0$ with $x_0 = 1$. The shaded side is where the node fires.

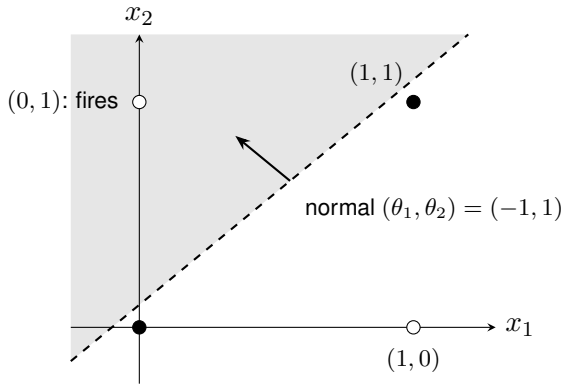


Figure 9: Hidden layer node 2 implements (**not** x_1) **and** x_2, i.e. $x_2 - x_1 > 0.1$. $\bar{\theta} = (-0.1,\ -1,\ 1)$.

Each node computes $\hat{y} = \sigma(\bar{x}^T\bar{\theta})$, a logistic regression. (We have yet to introduce notation to represent multiple layers.)

input		hidden layer		output
x_1	x_2	node 1	node 2	$\hat{y}$
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

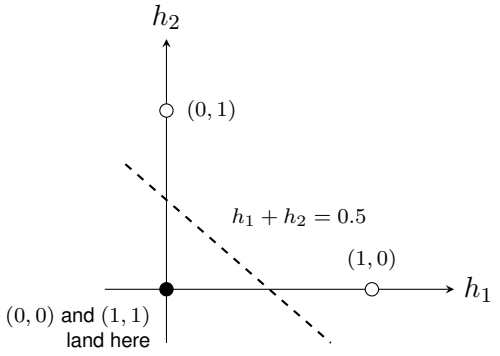


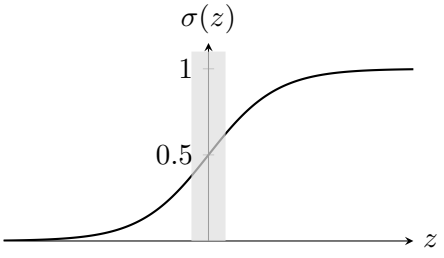
Figure 10: Truth table for the layers of "logistic regression" nodes (left), and the four inputs as seen by the output node (right). The hidden layer maps both class-0 inputs to the same point, so a single line, $h_1 + h_2 = 0.5$, now separates the classes.

This is why one hidden layer is enough: the hidden layer re-represents the inputs so that the output node, itself just a logistic regression, faces a linearly separable problem.

In practice, the weights above are too small, due to the steepness of the sigmoid (logistic function) when the weights are this small.

Let $z = \sum_{i=0}^{n} x_i \theta_i$ where $x_0 = 1$. The output from a logistic regression node would be

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$



In the shaded region, the change in z from < 0 to > 0 is smoothed out: $\sigma(0.1) = 0.525$ and $\sigma(-0.1) = 0.475$.

	input		hidden layer outputs		
	x_1	x_2	node 1	node 2	$\hat{y}$
logically $0, 0$ but $0.4, 0.4$ due to noise	0.4	0.4	0.475	0.475	0.6106
logically $0, 1$	0.4	0.6	0.426	0.525	0.6108
logically $1, 0$	0.6	0.4	0.525	0.426	0.6108
logically $1, 1$	0.6	0.6	0.475	0.475	0.6106

Figure 11: Results for inputs near the decision boundary on the input layer.

In Figure 11, we show what happens when we use the weights in Figure 7 with sigmoid nodes. The weights on the hidden layer are $[-0.1,\ 1,\ -1]$ and $[-0.1,\ -1,\ 1]$, and on the output node $[-0.5,\ 1,\ 1]$, each written as $[\theta_0,\ \theta_1,\ \theta_2]$. We show this more clearly in Figure 12.

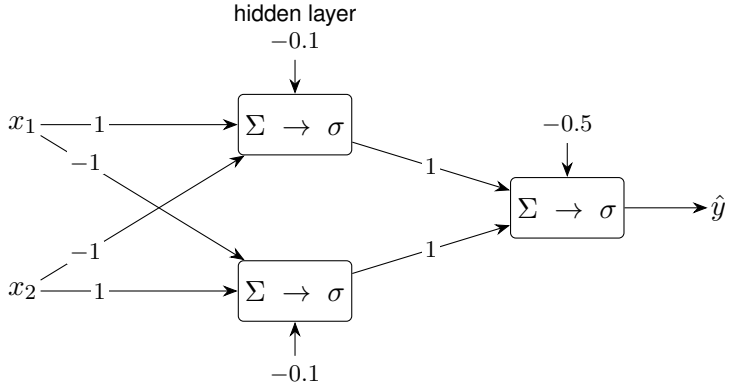


Figure 12: Showing each node in an XOR neural network. Each box sums its weighted inputs plus its bias (the short arrows), then applies the sigmoid.

With such small weights and bias values the neural network can produce ambiguous or even erroneous results from points near the decision boundaries on our input layer.

In Figure 11, the outputs are all around 0.5 and all somewhat above 0.5. If we apply a threshold of 0.5 on the output, all four cases resulted in a 1.

If we increase all the weights and biases proportionally, this does not move or reorient the decision boundary but it magnifies the input to the log σ the function.

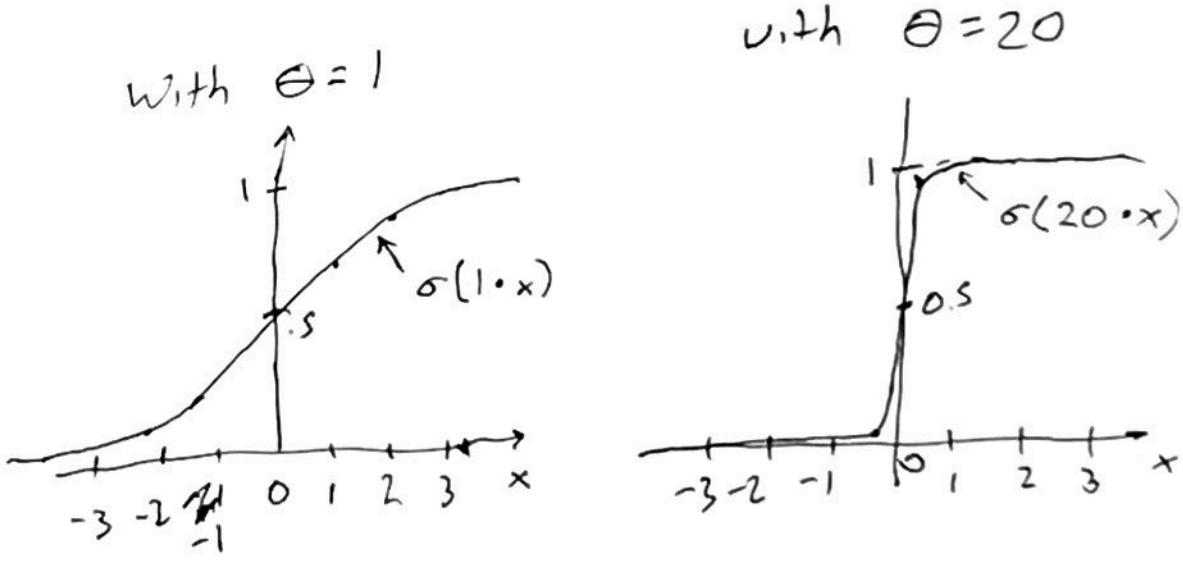


Figure 13: Heavier weights (more extreme parameters) result in a steeper, crisper boundary.

With steeper boundaries, the system learns more slowly, but it can better distinguish values that fall near the decision boundary.

input		hidden layer outputs		
x_1	x_2	node 1	node 2	$\hat{y}$
0.4	0.4	0.119	0.119	0.0053
0.4	0.6	0.00247	0.881	0.99953
0.6	0.4	0.881	0.00247	0.99953
0.6	0.6	0.119	0.119	0.0053

Figure 14: Same network, but with all weights and biases multiplied by 20.

With heavier weights, the boundary at 0.5 is sharper, allowing the neural network to clearly distinguish between 0.4 and 0.6.

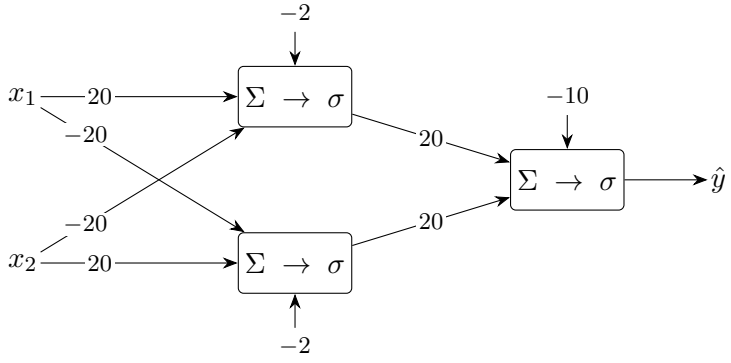


Figure 15: Neural network with the same weights as used to generate Figure 14.